

OBSERVATION TASK 2

Field Details

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	Task 2
Student Name	Nagadoleswari Madabathula
Roll No.	A24126552267
Date	30-08-2026

1. TITLE

Dynamic Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To create a JavaScript application that uses a class to represent a student, accepts user input through a form, and dynamically generates and displays the student's profile on the webpage using DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To define a JavaScript class with properties such as Name, Roll Number, Department and CGPA.
- To create an object of the class using user-provided values.
- To select DOM elements using getElementById().
- To handle a button click event.
- To dynamically create an HTML element using createElement() and set its content.
- To add the dynamically created element to the webpage using appendChild().
- To apply basic CSS styling to the form and the dynamically generated profile.

4. THEORY

A JavaScript class is a template for creating objects that bundle related data (properties) and behaviour together. The constructor() method runs automatically when a new object is created with the new keyword, and it is used to initialise the object's properties from the arguments passed to it.

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. document.getElementById() is used to select a specific element from this tree, while the .value property reads the current content of an input field.

An element can also be created entirely from JavaScript using document.createElement(), which returns a new, unattached element. Its content can be set using the innerHTML property, and it is inserted into the visible page using parentElement.appendChild(childElement).

Event handling connects user actions to code. An onclick attribute (or addEventListener) runs a function every time the button is clicked, allowing the page to react to user input without reloading.

In this experiment these concepts are combined: a Student class stores the entered details as an object, and a click event handler reads the input values, creates the object, and builds the profile display dynamically using DOM methods.

5. ALGORITHM / PROCEDURE

- Create an HTML file with input fields for Name, Roll Number, Department and CGPA, a "Create Profile" button, and an empty output container.
- Add internal CSS to style the form container and the profile output.
- Define a Student class with a constructor that accepts name, rollNumber, department and cgpa.
- Attach a click handler to the button using the onclick attribute.
- Inside the handler, read the values of all input fields using .value.
- Create a new Student object using the values read from the form.
- Select the output container element and clear any previous content.
- Dynamically create a <div> element for the profile using createElement().
- Set the profile's content using innerHTML with the student object's properties.
- Append the created element to the output container using appendChild().
- Open the HTML file in a browser, enter sample student details, and click the button.
- Verify that the profile is displayed correctly and repeat with different input values.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f2f2f2;
      text-align: center;
      padding: 30px;
    }

    .container {
      width: 400px;
      margin: auto;
      background: white;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0px 4px 10px gray;
    }
  </style>
</head>
<body>
```

```

input {
    width: 90%;
    padding: 10px;
    margin: 8px;
    border: 1px solid gray;
    border-radius: 5px;
}

button {
    padding: 10px 20px;
    background: green;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

.profile {
    margin-top: 20px;
    padding: 15px;
    background: #e8f5e9;
    border-radius: 8px;
    text-align: left;
}
</style>
</head>
<body>
    <div class="container">
        <h1>Student Profile</h1>
        <input type="text" id="name" placeholder="Enter Name">
        <input type="text" id="roll" placeholder="Enter Roll Number">
        <input type="text" id="department" placeholder="Enter Department">
        <input type="text" id="cgpa" placeholder="Enter CGPA">
        <br><br>
        <button onclick="createProfile()">Create Profile</button>
        <div id="output"></div>
    </div>
    <script>
        // Student Class
        class Student {
            constructor(name, rollNumber, department, cgpa) {
                this.name = name;
                this.rollNumber = rollNumber;
                this.department = department;
                this.cgpa = cgpa;
            }

```

```

    }

    // Function to create student profile
    function createProfile() {

        // Get user input values
        let name = document.getElementById("name").value;

        let rollNumber = document.getElementById("roll").value;

        let department = document.getElementById("department").value;

        let cgpa = document.getElementById("cgpa").value;

        // Create Student Object
        let student = new Student(
            name,
            rollNumber,
            department,
            cgpa
        );

        // Select output div
        let output = document.getElementById("output");

        // Clear previous output
        output.innerHTML = "";

        // Create profile div dynamically
        let profile = document.createElement("div");

        profile.className = "profile";

        // Add student details
        profile.innerHTML = `
            <h2>Student Profile</h2>
            <p><b>Name :</b> ${student.name}</p>
            <p><b>Roll No :</b> ${student.rollNumber}</p>
            <p><b>Department :</b> ${student.department}</p>
            <p><b>CGPA :</b> ${student.cgpa}</p>
        `;

        // Add profile to webpage
        output.appendChild(profile);
    }

</script>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>class Student</code>	Defines a blueprint for a student object with name, rollNumber, department and cgpa properties.
<code>constructor()</code>	Initializes a new Student object's properties when it is created with new.
<code>document.getElementById()</code>	Selects a single DOM element by its id attribute, e.g. an input field or output container.
<code>.value</code>	Reads the current text entered by the user in an input field.
<code>onclick="createProfile()"</code>	Registers a function to run whenever the button is clicked (event handling).
<code>new Student(...)</code>	Creates an object of the Student class using the values read from the form.
<code>document.createElement()</code>	Creates a new HTML element (a <div>) that is not yet part of the page.
<code>.innerHTML</code>	Sets the HTML content inside the profile <div>, building the labelled Name / Roll No / Department / CGPA lines.
<code>appendChild()</code>	Inserts the created profile element into the page as a child of the output container.
<code>output.innerHTML = ""</code>	Clears any previously displayed profile before showing the new one.

8. TEST CASES AND EXECUTION DATA

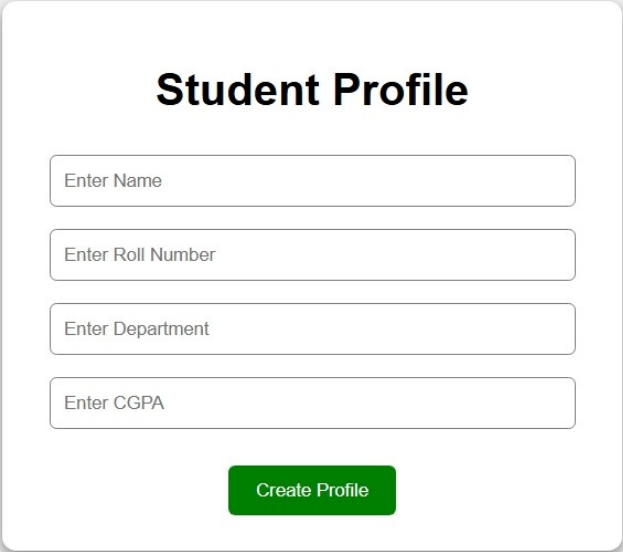
Each test case was executed by entering the given values into the form and clicking the "Create Profile" button.

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Status
TC-01	Nagadoleswari Madabathula	A24126552267	CSM	8.09	Profile displayed with entered details	Pass
TC-02	Rahul Verma	A24126552011	CSE (AI & ML)	7.85	Profile updates to new details	Pass
TC-03	Sneha Iyer	A24126552022	ECE	9.02	Profile displayed correctly	Pass
TC-04	Mohit Rao	A24126552033	CSE (Cyber Security)	0.00	Profile displayed with 0.00 CGPA	Pass

Additional Validation Test

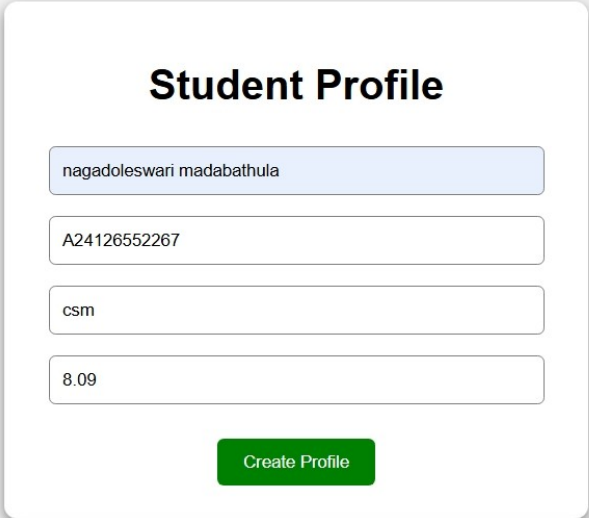
Test Case	Input	Expected Result	Status
TC-05	All fields left empty, button clicked	Profile is generated with blank values (no fields are marked required)	Pass

9. OUTPUT



A screenshot of a web form titled "Student Profile". The form is white with rounded corners and a subtle shadow, set against a light gray background. It contains four input fields stacked vertically, each with a placeholder text: "Enter Name", "Enter Roll Number", "Enter Department", and "Enter CGPA". Below these fields is a green button with the text "Create Profile".

Output 1: Student Profile Form (blank, before entering details)



A screenshot of the same "Student Profile" form, now filled with student details. The input fields contain the following text: "nagadoleswari madabathula" (highlighted in light blue), "A24126552267", "csm", and "8.09". The green "Create Profile" button remains at the bottom.

Output 2: Form filled with student details, before clicking "Create Profile"

Student Profile

nagadoleswari madabathula

A24126552267

csm

8.09

Create Profile

Student Profile

Name : nagadoleswari madabathula

Roll No : A24126552267

Department : csm

CGPA : 8.09

Output 3: Generated Profile displayed after clicking "Create Profile"

10. RESULT

Thus, a JavaScript application was successfully developed that defines a Student class, creates an object from user-entered form data, and dynamically generates and displays the student's profile on the webpage using DOM manipulation and event handling. The application was verified using multiple test cases.